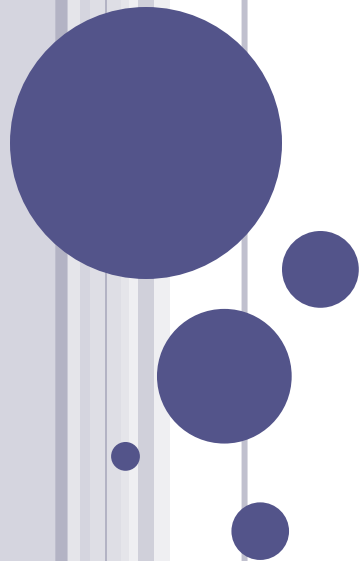


CHAPITRE1: ENVIRONNEMENT DISTRIBUÉ: CONCEPTS DE BASE



MDPS2

CONTEXTE ET MOTIVATIONS

- Scénario1: L'entreprise qui se développe et souhaite ouvrir des filiales à travers le monde
 - ➔ Cas de données jusqu'alors centralisées doivent être réparties sur différents sites
 - Comment découper logiquement ma base pour répartir les n-uplets sur les différents sites?
- Scénario2: L'entreprise A qui rachète l'entreprise B et qui souhaite faire du *reporting* basé sur l'ensemble des données du groupe.
 - ➔ Cas des données réparties sur différents sites qui doivent être interrogeables depuis n'importe quel site
 - Comment offrir à l'utilisateur un moyen d'accès transparents à toutes ces données ?

CONTEXTE ET MOTIVATIONS

- Scénario3: L'entreprise qui voit la quantité de ses données croître de manière exponentielle avec des latences de plus en plus importantes dans l'interrogation des données
 - ➔ Cas de traitement de masse de données épuisant les ressources du serveur
 - Comment garantir un temps de traitement satisfaisant malgré la quantité croissante de données?



BD Distribuées

DÉFINITIONS

- **Base de données distribuée (DDB) :**
- Collection de BD logiquement *reliées et physiquement distribuées sur un réseau*
- une grande quantité de données résidant sur plusieurs machines
- **SGBD Distribué :**
- Système logiciel de gestion rendant les applications insensibles à la distribution des données
- Permettant d'avoir un point d'entrée unique sur une base de données distribuées (TRANSPARENCE)
- *Base de données distribuée N'EST PAS :*
 - **Fichiers indépendants sur des sites différents,**
 - **Base centralisée accessible par différentes applications**

EVOLUTION DES SI

MainFrame

Les SGBD des débuts : **centralisés**

- Pluri-utilisateurs
- Terminaux

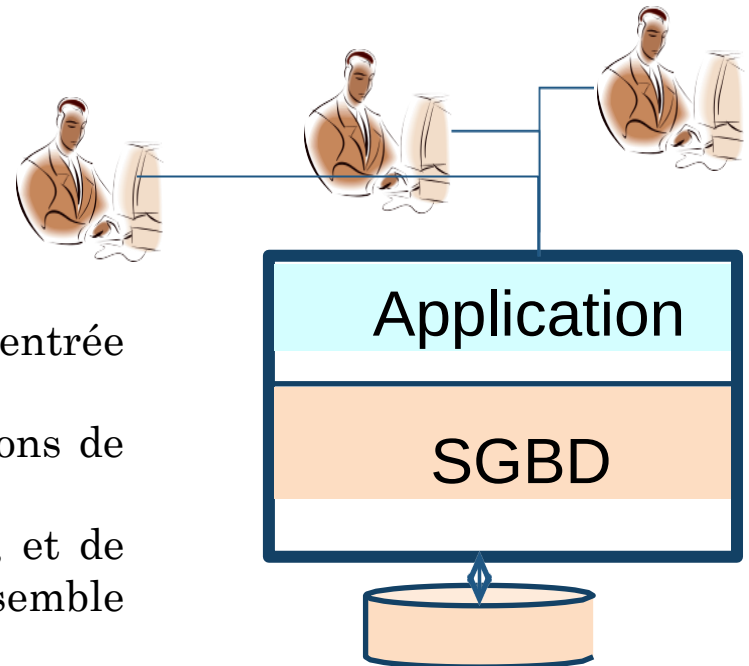
Toute l'informatique des entreprises était concentrée dans un serveur unique : le mainframe.

Le mainframe supporte en général des applications de gestion essentielles pour l'entreprise.

→ toute la logique de persistance, de traitement, et de présentation est agrégée dans un ensemble indissociable.

Avantage: système totalement cohérent et fiable (sous contrainte d'assurer la haute disponibilité)

Inconvénient: ses coûts d'acquisition et d'exploitation sont élevés



EVOLUTION DES SI

Architecture Client-Serveur

Client

- Application
- Interface graphique

Serveur

- Gestion des requêtes
- Gestion des transactions
- Gestion des pannes...

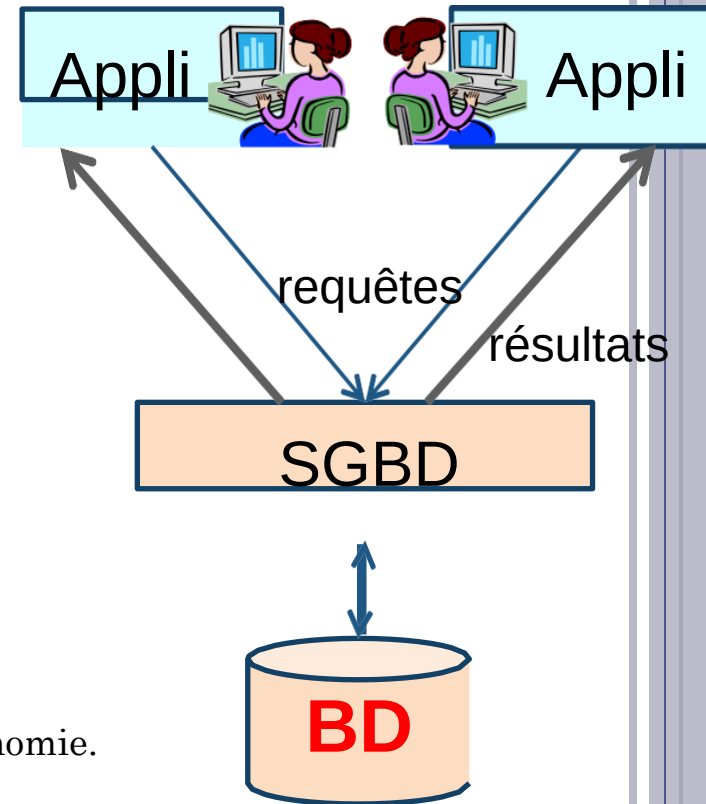
Avantages:

plus de graphisme et plus de sophistication en terme d'ergonomie.
faible coût

Inconvénients:

L'absence de gouvernance de cette croissance a souvent abouti à des duplications d'informations, saisies et gérées de manière autonome par ces différents services. Les applications client/serveur ont engorgé les PCs de multiples exécutables difficiles à maintenir et dotés d'interfaces incohérentes.

→ Cette multiplication s'est révélée complexe à maîtriser et donc improductive pour l'utilisateur



EVOLUTION DES SI

Architecture 3-tiers

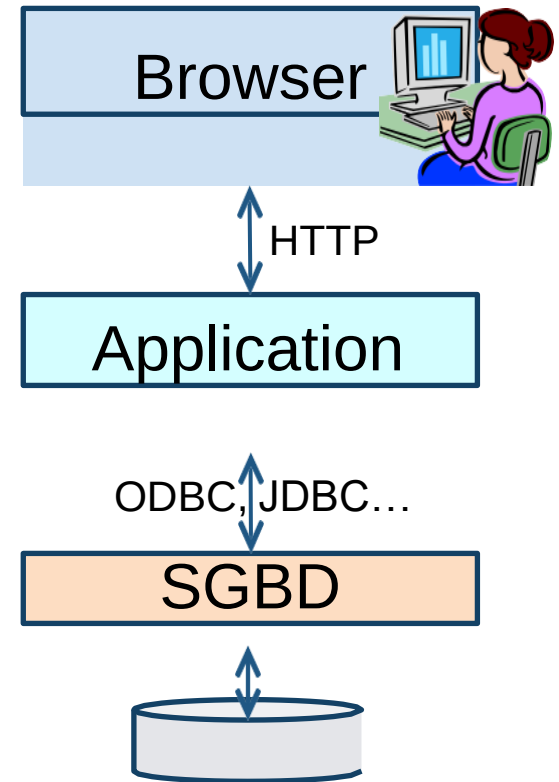
Le niveau client :

- un navigateur Web
- Présente du contenu, par ex HTML

Le niveau intermédiaire

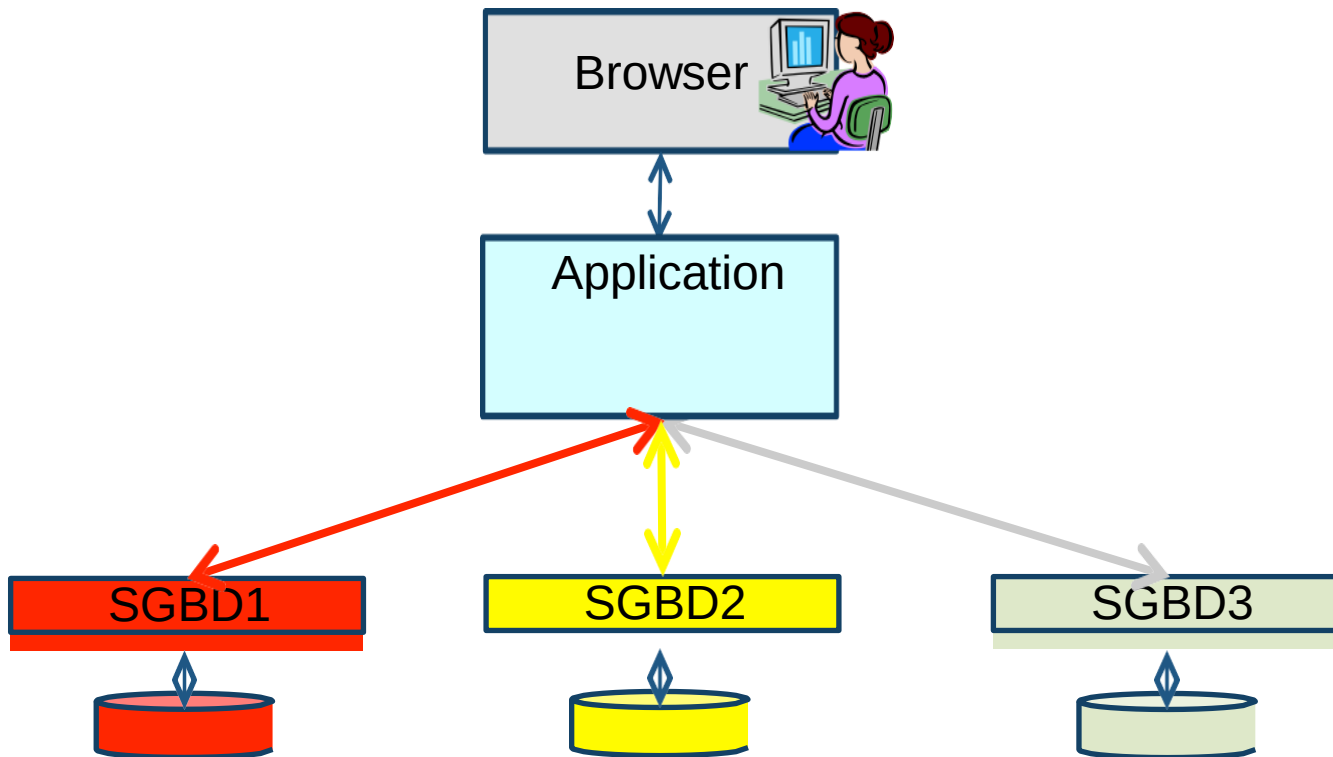
- Communique avec le SGBD
- Gère l'application (Java, C++, C# ...)
- Génère le contenu pour le client

Le niveau serveur : base de données



EVOLUTION DES SI

Vers la distribution : plusieurs serveurs de données



POURQUOI DISTRIBUER

○ améliorer les performances → BD Réparties (fragmentées)

- On part d'une grosse base de données
- On la distribue en fragments pour améliorer les performances
- On obtient une base de données distribuées

Un ordinateur et un disque

- 1 téraoctet
- Scan séquentiel
- 166 minutes (> 2.5 heures)

100 disques en parallèle

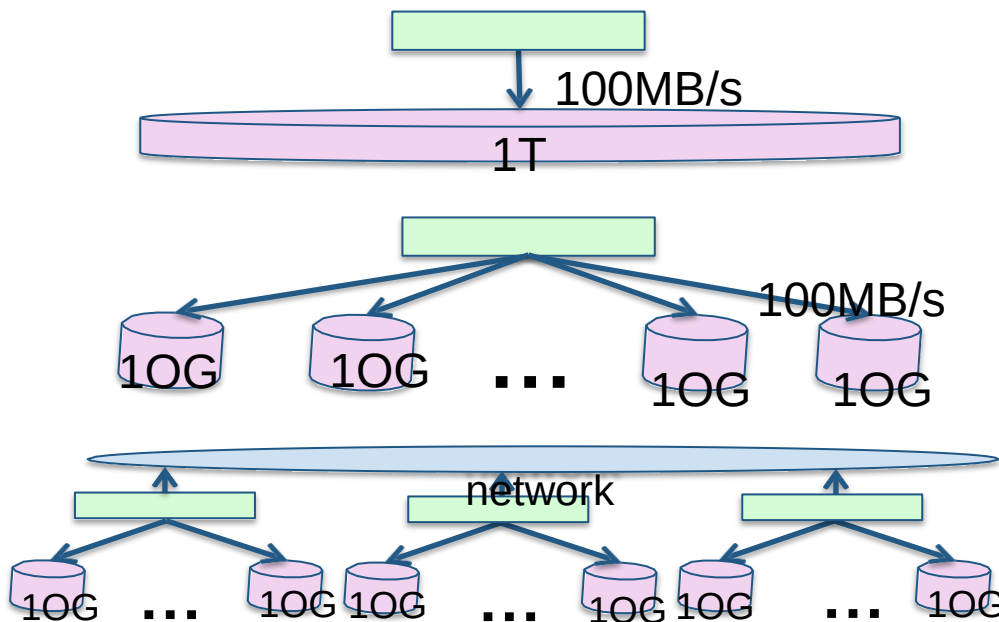
moins de 2mn

100 ordinateurs distribués

Chacun son propre CPU

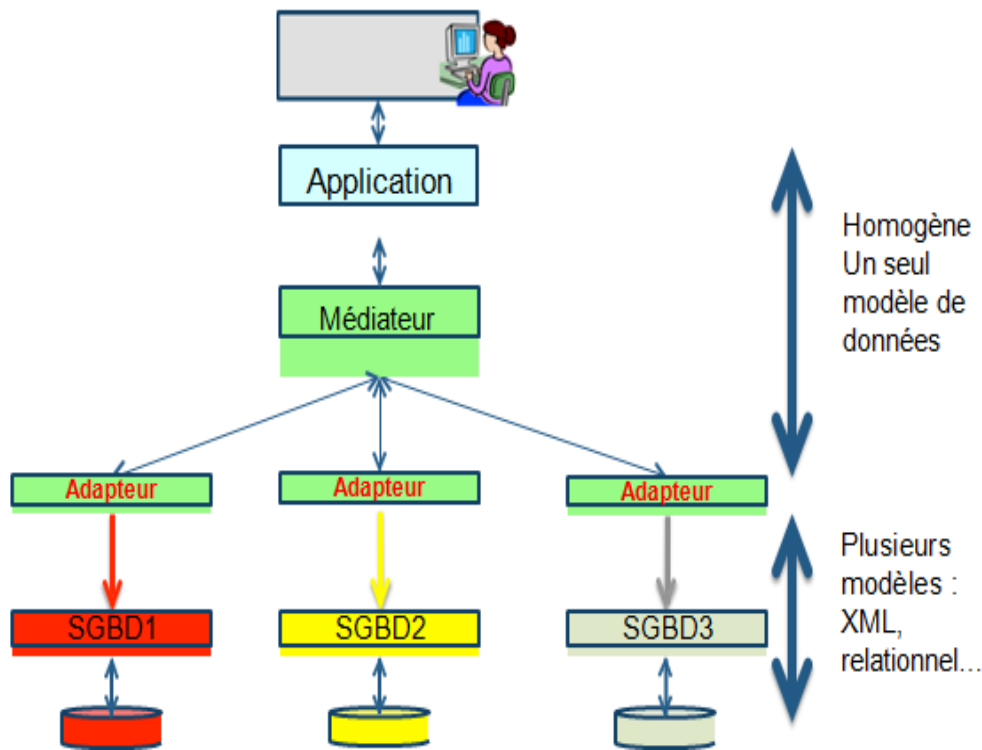
Chacun son disque

Ça passe à l'échelle



POURQUOI DISTRIBUER

- Conserver l'autonomie des différents systèmes (BD Fédérées) :
- Tendance dans l'industrie : des structures de moins en moins hiérarchiques, de plus en plus autonomes
- On part de bases de données existantes (autonomes)
- On les intègre pour obtenir un point d'entrée unique
- On obtient une base de données distribuées



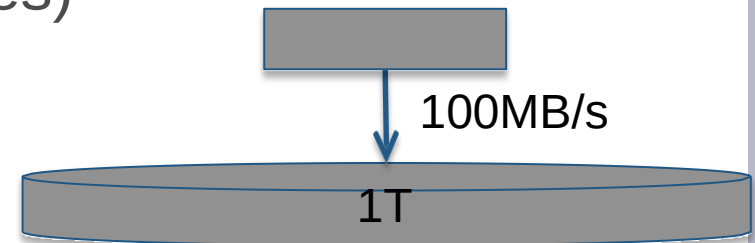
- La médiation permet d'accéder simplement à des données hétérogènes
- Sans perturber le fonctionnement des SGBD opérationnels

POURQUOI DISTRIBUER

- Pour plus de sûreté (BD répliquées)

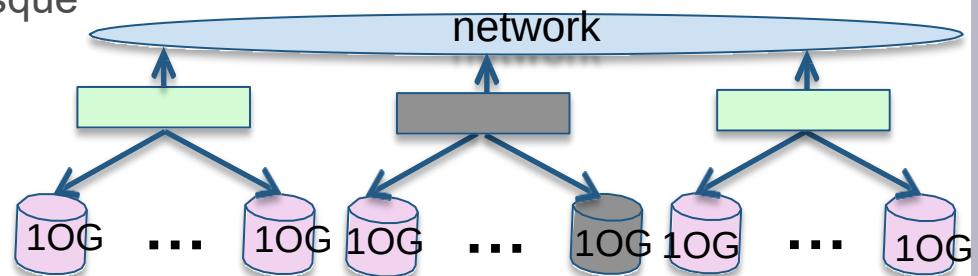
- **Mono-serveur**

- Panne du processeur ou du disque
- Arrêt du service
- Perte des données



- **Multiserveurs**

- Panne d'un processeur ou d'un disque
- Le service continue
- Grâce à la réplication,
- pas de perte des données



PRINCIPES FONDAMENTAUX

« To the user, a distributed system should look exactly like a nondistributed system. » (C. Date, Introduction to Database Systems)

○ **Autonomie Locale**

- La BD locale est complète et autonome (intégrité, sécurité, gestion), elle peut évoluer indépendamment des autres (upgrades...)

○ **Egalité entre sites**

- Un site en panne ne doit pas empêcher le fonctionnement des autres sites (mais perturbations possibles)

○ **Fonctionnement continu**

- La distribution permet une résistance aux fautes et aux pannes (en théorie)

PRINCIPES FONDAMENTAUX

- **Transparence de localisation**

- l'utilisateur n'a pas savoir où sont les données
- Accès uniforme aux données quel que soit leur site de stockage

- **Transparence de réseau/système**

- Il n'a pas à connaître les aspects techniques du réseau
- Le SGBD est accessible à travers les différents types de réseau utilisés

- **Transparence de fragmentation**

- il n'a pas à connaître comment les données sont fragmentées
- Des données (d'une même table) éparpillées doivent être vues comme un tout

- **Transparence de réplication**

- Les données répliquées doivent être maintenues en cohérence (délai possible)
- il n'a pas à savoir comment/si elles sont répliquées

PRINCIPES FONDAMENTAUX

- **Requêtes distribuées**

- L'exécution d'une requête peut être répartie (automatiquement) entre plusieurs sites (si les données sont réparties)

- **Transactions réparties**

- Le mécanisme de transactions peut être réparti entre plusieurs sites

- **Indépendance vis-à-vis du matériel**

- Le SGBD fonctionne sur les différentes plateformes utilisées

PRINCIPES FONDAMENTAUX

- **Indépendance vis-à-vis du SE**
 - Le SGBD fonctionne sur les différents SE...
- **Indépendance vis-à-vis du SGBD**
 - La base peut être distribuée sur des SGBD hétérogènes (En théorie)...

AVANTAGES DE LA RÉPARTITION DES BASES DE DONNÉES

- Architecture plus adaptée à l'organisation des entreprises décentralisées.
- Plus de fiabilité : les bases de données réparties ont souvent des données répliquées.
 - La panne d'un site n'est pas très importante pour l'utilisateur, qui s'adressera à autre site.
- Meilleures performances :
 - réduire le trafic sur le réseau est une possibilité d'accroître les performances.
 - rapprocher les données de l'endroit où elles sont accédées.
- Répartir la charge sur les processeurs et sur les entrées/sorties.
- Faciliter l'accroissement: l'accroissement se fait par l'ajout de machines sur le réseau.

INCONVÉNIENTS

- Coût :
 - la distribution entraîne des coûts supplémentaires en terme de communication, et en gestion des communications (hardware et software à installer pour gérer les communications et la distribution).
- Sécurité :
 - la sécurité est un problème plus complexe dans le cas des bases de données réparties que dans le cas des bases de données centralisées.
- Complexité
 - Rajoute des couches de logiciel
 - Communication
 - Distribution du contrôle
 - Conception de la base de données
- Incohérence
 - Gestion des contraintes d'intégrité et de cohérence

CONCEPTION DE BD RÉPARTIES

- On ne met en place une BD répartie qu'en cas de réel besoin
 - Démarche de conception délicate
 - Gestion complexe
 - L'évolution du SI peut invalider la solution retenue...
- Des raisons valables :
 - Volumes de données,
 - sites distants, etc.
 - Fusions de SI

CONCEPTION DE BD RÉPARTIE

- Décision à prendre en fonction de critères techniques et organisationnels :
 - minimiser le nombre de transferts entre sites,
 - Minimiser les temps de transfert,
 - Minimiser le volume de données transférées,

CONCEPTION DE BD RÉPARTIE

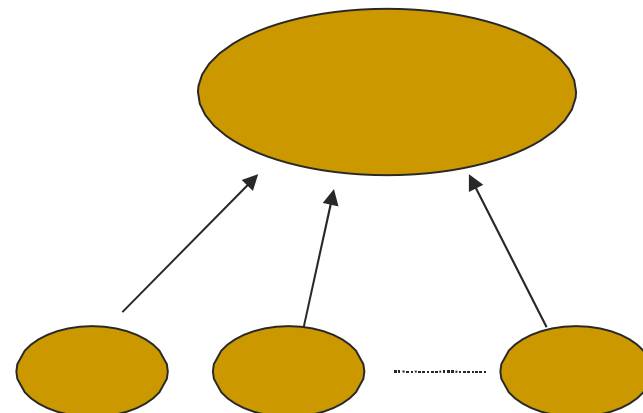
- **Schéma local**
- Une base de données locale comporte un schéma géré par le SGBD local.
- Lors de la constitution d'une base de données repartie, chaque base locale rend visible une partie de la base aux sites clients.
- **Schéma global**
- Comme toute base de données, une base de données repartie possède un schéma appelé schéma global qui permet de définir l'ensemble des types de données de la base.
- Le schéma global ignore les concepts d'implémentation.
 - il est souvent appelé schéma conceptuel.
- Dans une base de données repartie, le schéma global n'est pas forcément matérialisé.
- Chaque base locale en implémente une partie. Ces parties locales sont les seules matérialisées sur les disques.

CONCEPTION D'UNE BASE DE DONNÉES RÉPARTIE

- **Méthode de conception**
- Il existe deux approches pour la conception d'une base de données répartie selon la relation schéma global, schéma local. :
 - Conception ascendante (*bottom up design*)
 - Conception descendante (*top down design*)

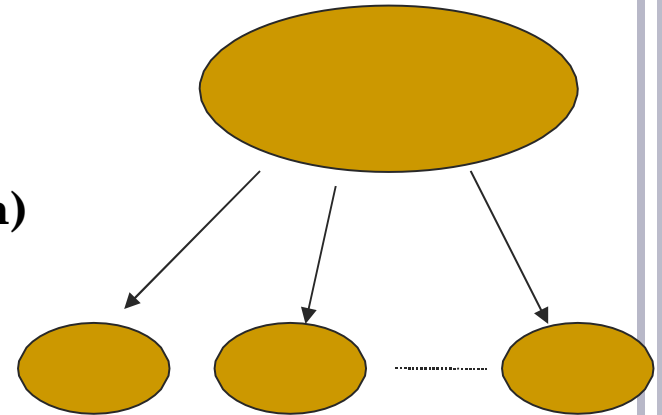
CONCEPTION D'UNE BASE DE DONNÉES RÉPARTIE

- **Conception (Approche) ascendante : (Bottom up design)**
- on part de l'existant (Distribution pré-existante).
- L'objectif principal est d'intégrer les bases locales dans schéma global.
- Elle nécessite une consolidation, uniformisation :
 - **1 Réconciliation sémantique**
 - **2 Identifier les données semblables**
 - **3 Accorder leurs types, gérer leur cohérence...**
 - **4 Interfacer ou adapter les SGBD...**



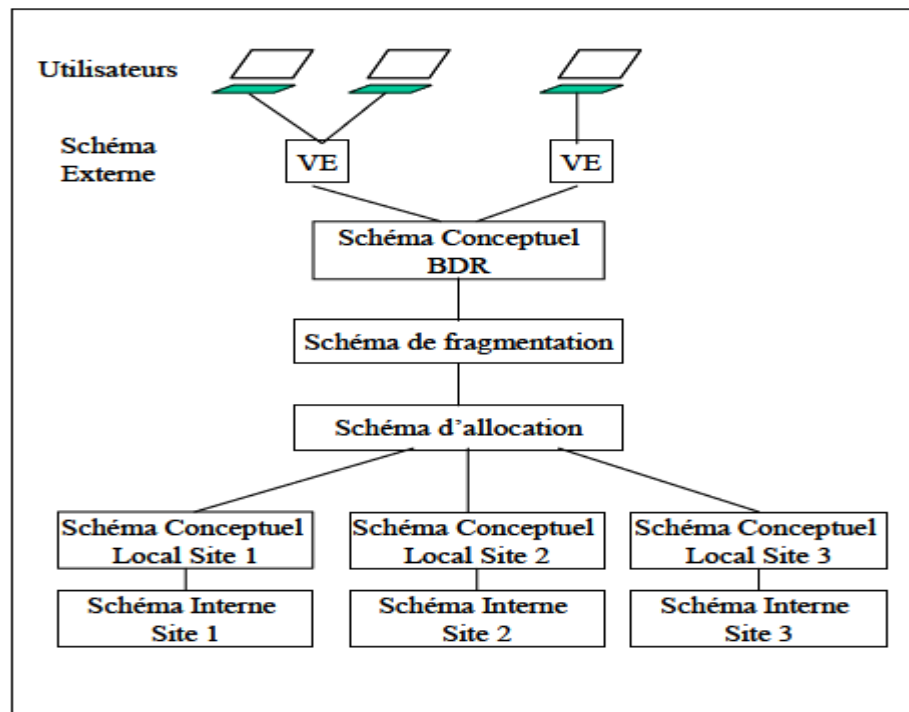
CONCEPTION D'UNE BASE DE DONNÉES RÉPARTIE

- **Conception descendante : (Top down design)**
- On part du schéma global
- On le scinde en schémas locaux
- On part de zéro (nouvelle base)
- Recherche de performance (pas forcément de répartition géographique)
- *L'approche top down est intéressante quand on part du néant. Si les BDs existent déjà la méthode bottom up est utilisée*
- Les points suivants sont présents dans cette approche :
 - 1 Conception du schéma conceptuel global**
 - 2 Distribution pour obtenir des schémas conceptuels locaux**
 - 3 Les tables du schéma global sont fragmentées (processus de fragmentation)**
 - 4 Les fragments sont donc placés sur des sites (processus d'allocation)**



CONCEPTION D'UNE BASE DE DONNÉES RÉPARTIE

Conception descendante (*top down design*)



Architecture d'une BDR.

SCHÉMAS D'UNE BD RÉPARTIE

- Dans un contexte centralisé :

Analyse des besoins



Modèle conceptuel de données



Modèle logique

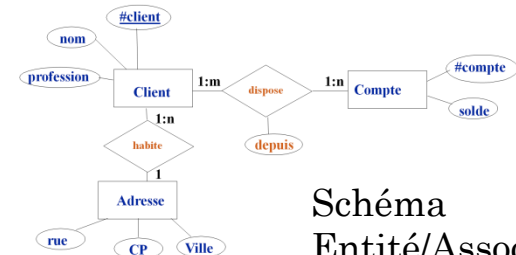


Schéma
Entité/Association

Modèle relationnel

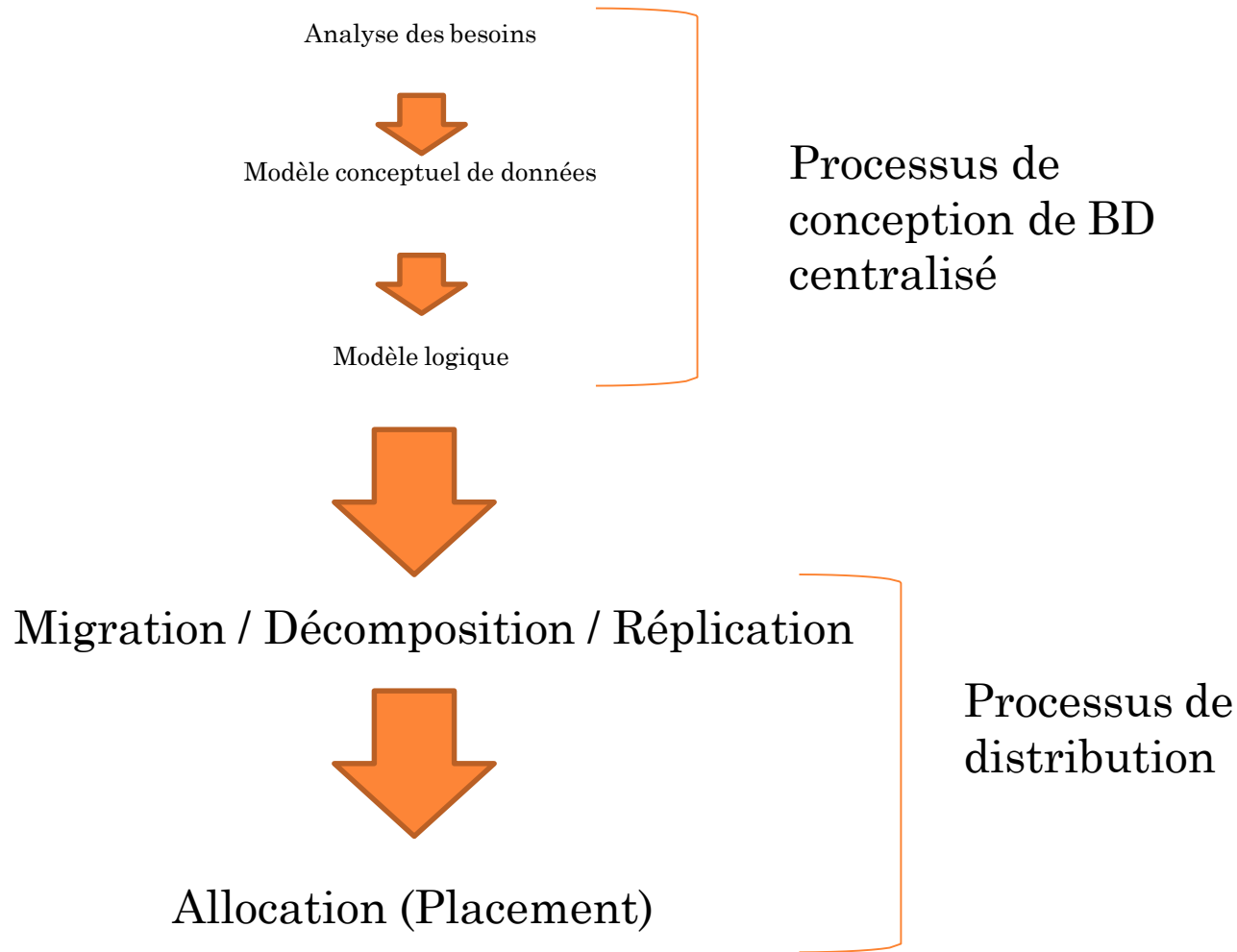
Acteur		
Nom	Prénom	Nationalité
Cruz	Penelope	ESP
Maura	Carmen	ESP
Johansson	Scarlett	USA
Villeret	Jacques	FR
Lhermite	Thierry	FR
Downey	Robert	UK
Allen	Woody	USA

Réalisateur		
Nom	Prénom	Pays
Almodovar	Pedro	ESP
Allen	Woody	USA
Veber	Francis	FR
Favreau	John	USA

Artiste = Acteur $\cup$ Réalisateur

Nom	Prénom	Nationalité
Cruz	Penelope	ESP
Maura	Carmen	ESP
Johansson	Scarlett	USA
Villeret	Jacques	FR
Lhermite	Thierry	FR
Downey	Robert	UK
Almodovar	Pedro	ESP
Allen	Woody	USA
Veber	Francis	FR
Favreau	John	USA

CONTEXTE DÉCENTRALISÉ

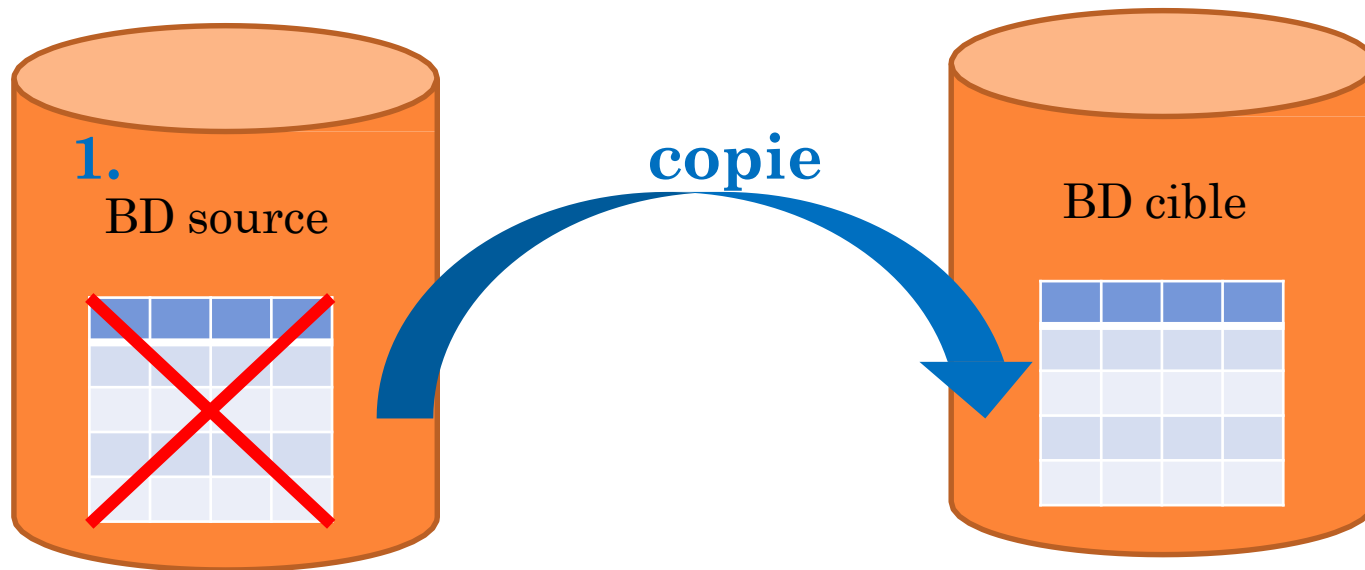


CONTEXTE DÉCENTRALISÉ

- Le schéma global est constitué :
 - d'un schéma conceptuel global
 - Contenant la description globale et unifiée de toutes les données de la BDR
 - Indépendant de la répartition des données
 - d'un schéma de placement (d'allocation)
 - Contenant les règles de correspondance avec les données locales
 - indépendant à la fragmentation et à la réplication

MIGRATION

- Transfert d'une relation complète sur un site distant

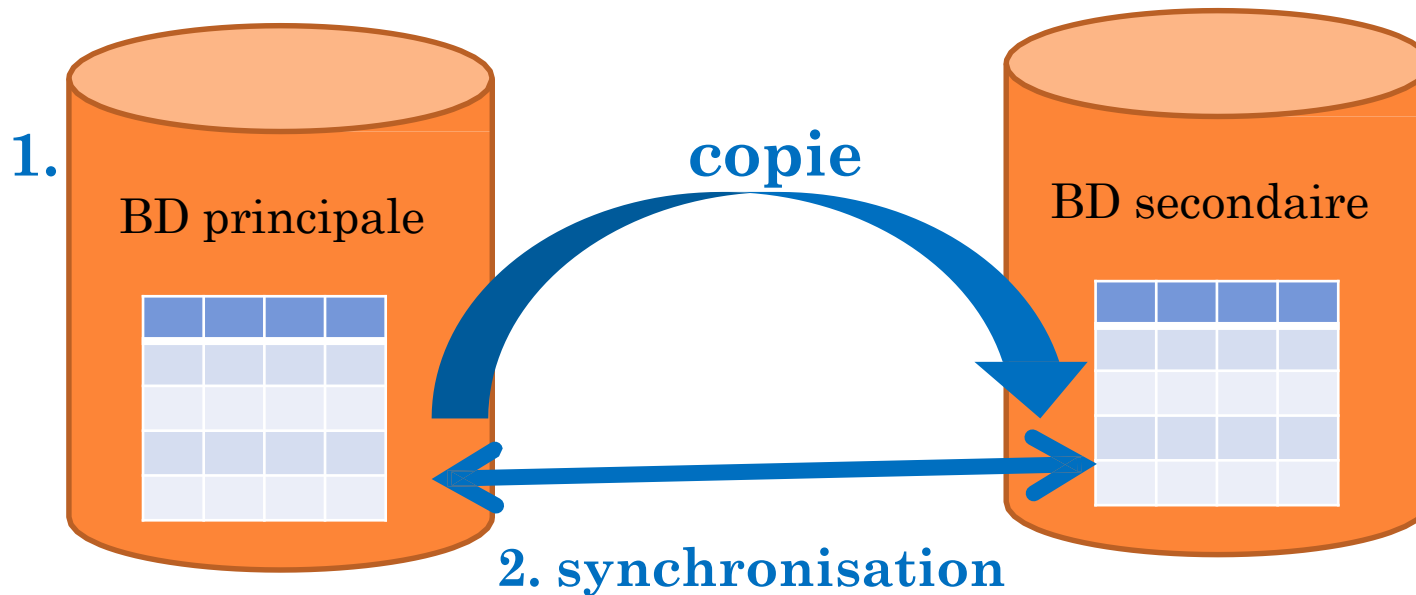


2.suppression

- Intérêt
 - Rapprocher les données des besoins

RÉPLICATION/DUPLICATION

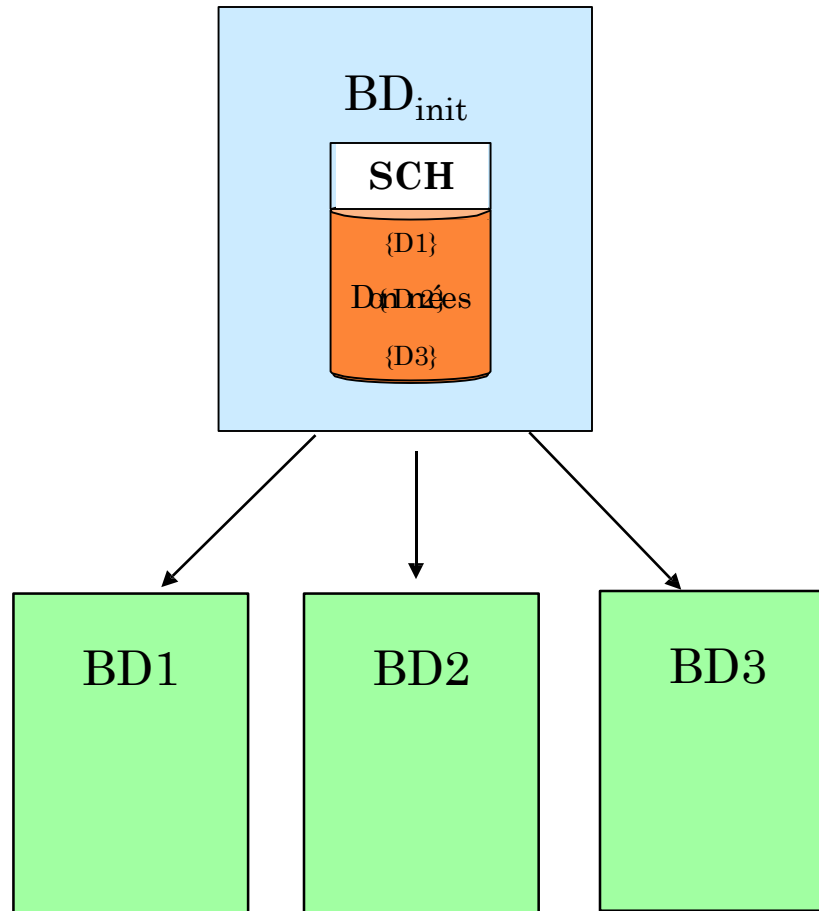
- Création d'une copie conforme d'une table (ou ensemble de tuples) sur un site distant. La copie doit rester cohérente avec les données sources



- **Intérêt**

- Avoir les mêmes données sur différents sites

FRAGMENTATION/DÉCOMPOSITION



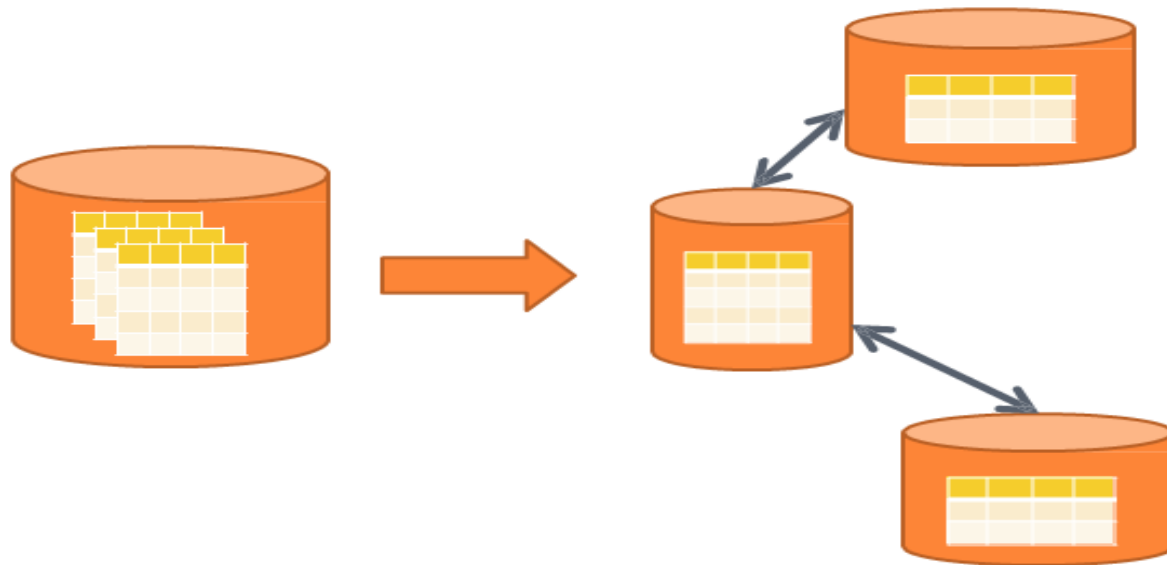
SCH est le schéma global qui permet l'interrogation des données stockées sur BD1, BD2 et BD3

○ Fragmentation



DÉFINITION DE LA FRAGMENTATION

- .La fragmentation est le processus de décomposition d'une base de donnée logique en un ensemble de "sous" bases de données. Cette décomposition doit être sans perte d'information.



LES RÈGLES DE FRAGMENTATION

- **Les règles à appliquer sont :**
- 1. La complétude : pour toute donnée d'une relation R , il existe un fragment R^i de la relation R qui possède cette donnée.
- 2. La reconstruction : pour toute relation décomposée en un ensemble de fragments R_i , il existe une opération de reconstruction.
- 3. La Disjonction: assure que les fragments d'une relation sont disjoints deux à deux. chaque élément de R (hormis les clés) ne doit pas être dupliqué



INTERET DE LA FRAGMENTATION

- **Exemple**

- Relation Employé

E(enum, nom, site, salaire,...)

- Un employé n'appartient qu'à un seul site

- 40 000 employés à Paris, 40 000 à Lyon, et 20 000 à Marseille

- Charge de travail (workload)

- 90% des requêtes à Paris/Lyon/Marseille sont sur des employés locaux

INTÉRÊT DE LA FRAGMENTATION

- Pour illustrer
- Accès disque = 1 unité & Accès réseau = 10 unités
- **Sans fragmentation** (un SGBD à Paris)

Coût

- à Paris = 1 unité
- à L/M = 10
- moyen = $0.4 + 0.6 * 10 = 6.4$ unités

- **Avec fragmentation** (3 SGBD : BD-P, BD-L, BD-M)

Coût

- – à P/L/M = $0.9 * (0.4 + 0.4 + 0.2) * 1 + 0.1 * 10 = 1.9$
 - ↻ moyen = 1.9 unités
 - ↻ Evite le risque de saturer le SGBD de Paris

TECHNIQUES DE FRAGMENTATION

- Il existe plusieurs techniques de fragmentation:
 - Fragmentation horizontale
 - Primaire
 - Dérivée
 - Fragmentation verticale
 - Fragmentation mixte



FRAGMENTATION HORIZONTALE

- **Fragmentation horizontale primaire :**
- La Fragmentation horizontale est définie par l'opération de sélection
- **Exemple**
 - Client (NCL, Nom, Ville) peut être fragmentée :
 - Client1= SELECT * FROM Client WHERE Ville = "Paris"
 - Client2= SELECT * FROM Client WHERE Ville <> "Paris"
- **Reconstruction de la relation initiale:**
- **Client = Client1 $\cup$ Client2**

Client	NCL	Nom	Ville
	C1	Dupont	Paris
	C2	Durant	Lille
	C3	Martin	Nice
	C4	Olivier	Paris

Client1	NCL	Nom	Ville
	C1	Dupont	Paris
	C4	Olivier	Paris

Client2	NCL	Nom	Ville
	C2	Durant	Lille
	C3	Martin	Nice



FRAGMENTATION HORIZONTALE

- Fragmentation horizontale dérivée :
- La Fragmentation d'une table en fonction des fragments horizontaux d'une autre table.
- Exemple
- Commande (NCL, N°Produit, Date, Qte, N°Représentant)

Commande1= SELECT * FROM Commande

WHERE NCL IN

(SELECT NCL FROM CLIENT1)

Commande2= SELECT * FROM Commande

WHERE NCL IN

(SELECT NCL FROM CLIENT2)

Cmd	NP	NCL	Date	Qte
	P1	C1	1/1/99	10
	P2	C2	1/1/99	20
	P2	C2	2/1/99	30
	P3	C4	4/1/99	30

Cmd1	NP	NCL	Date	Qte
	P1	C1	1/1/99	10
	P3	C4	4/1/99	30

Cmd2	NP	NCL	Date	Qte
	P2	C2	1/1/99	20
	P2	C2	2/1/99	30

Reconstruction de la relation initiale:
Commande = Commande1 $\cup$ Commande2

FRAGMENTATION VERTICALE

- La fragmentation verticale est le découpage d'une table en sous-tables par des projections permettant de sélectionner les colonnes qui composent chaque fragment → Répartition des attributs
- La fragmentation verticale est obtenue par décomposition de la table en groupes de colonnes.
- Toutes les valeurs des occurrences pour un même attribut se trouvent dans le même fragment
- L'opérateur de partitionnement est la projection (Π)
- L'opérateur de recomposition est la jointure ($\bowtie$)



FRAGMENTATION VERTICALE

- **Exemple**

- Soit le partitionnement de la relation *Client* en deux relations :

Relation Client

No client	Nom client	Prénom	Age
174723	Villard	Jean	29
177498	Cattell	Blaise	38
201639	Tsellis	Alan	51
203446	Kowalsky	Valdimir	36

- $\text{Cli1} = \pi[\text{NoClient}, \text{NomClient}] \text{ Client}$
- et $\text{Cli2} = \pi[\text{Noclient}, \text{Prénom}, \text{Age}] \text{ Client}$

Client (N°Client, Nom, Prénom, Age) peut être fragmentée :

$\text{Client1} = \text{SELECT N°Client, Nom FROM Client}$

$\text{Client2} = \text{SELECT N°Client, Prénom, Age FROM Client}$

NoClient	NomClient
174 723	Villard
177 498	Cattell
201 639	Tsellis
203 446	Kowalsky

Relation Cli1

NoClient	Prénom	Age
174 723	Jean	29
177 498	Blaise	38
201 639	Alan	51
203 446	Vladimir	36

Relation Cli2

La relation d'origine est obtenue avec la recombinaison suivante :

$\text{Client} = \text{Cli1} \bowtie \text{Cli2}$



FRAGMENTATION VERTICALE

Remarque importante :

« Il est important de conserver les clés dans chaque fragment vertical »

Sinon reconstruction impossible !

Cde

ncde	nclient	produit	qté
D 1	C 1	P 1	10
D 2	C 1	P 2	20
D 3	C 2	P 3	5
D 4	C 4	P 4	10

Cde1

ncde	nclient
D 1	C 1
D 2	C 1
D 3	C 2
D 4	C 4

Cde2

produit	qté
P 1	10
P 2	20
P 3	5
P 4	10

RÉPARTITION DES VALEURS (FRAGMENTATION HYBRIDE)

- C'est la combinaison des deux fragmentations précédentes, horizontale et verticale.
- Les occurrences et les attributs peuvent donc être répartis dans des partitions différentes.
- L'opération de partitionnement est une combinaison de projections et de sélections.
- • L'opération de recomposition est une combinaison de jointures et d'unions.



FRAGMENTATION MIXTE

Fragments définis par
sélection et projection

Client

nclient	nom	ville
C 1	Dupont	Paris
C 2	Martin	Lyon
C 3	Martin	Paris
C 4	Smith	Lille

Fragmentation horizontale

Client1

nclient	nom	ville
C 1	Dupont	Paris
C 3	Martin	Paris

Client2

nclient	nom	ville
C 2	Martin	Lyon
C 4	Smith	Lille

Fragmentation verticale

Client11

nclient	nom
C 1	Dupont
C 3	Martin

Client12

nclient	ville
C 1	Paris
C 3	Paris

Fragmentation verticale

Client21

nclient	nom
C 2	Martin
C 4	Smith

Client22

nclient	ville
C 2	Lyon
C 4	Lille

FRAGMENTATION MIXTE

- Relation $Cli11 \pi[NoClient, Nom] (\sigma[Ville=Paris] Client)$
- Relation $Cli12 \pi[NoClient, Ville] (\sigma[Ville=Paris] Client)$
- Relation $Cli21 \pi[NoClient, Nom] (\sigma[Ville \neq Paris] Client)$
- Relation $Cli22 \pi[NoClient, Ville] (\sigma[Ville \neq Paris] Client)$

- La relation $Client$ est obtenue avec : $(Cli11 \bowtie Cli12) \cup (Cli21 \bowtie Cli22)$



- Allocation des fragments



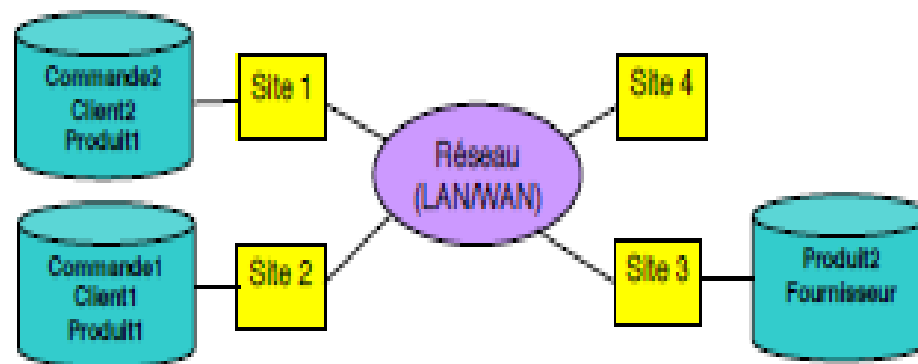
ALLOCATION DE FRAGMENTS AUX SITES

○ Non-Dupliquée :

- partitionnée
- chaque fragment n'existe que sur un seul site

○ Dupliquée :

- chaque fragment est sur plus d'un site
- maintien de la **cohérence des copies multiples**



PROCESSUS D'ALLOCATION DES FRAGMENTS.

- L'affectation des fragments sur les sites est décidée en fonction de l'origine prévue des requêtes qui ont servi à la fragmentation.
- Le but est de placer les fragments pour minimiser les transferts de données entre les sites.
- L'allocation peut se faire avec **réplication** ou **sans réplication**.
- La **réplication** favorise les performances des requêtes et augmente la disponibilité des données, mais est coûteuse en mise à jour des différents fragments.



PROBLÈME D'ALLOCATION:

○ Entrées:

- □ $F = \{F1, F2, \dots, F_n\}$ - ensemble de fragments
- □ $S = \{S1, S2, \dots, S_m\}$ - ensemble de sites
- □ $Q = \{Q1, Q2, \dots, Q_l\}$ - ensemble de requêtes
- **$\Rightarrow$ Le problème d'allocation consiste à trouver une distribution optimale de F sur S afin d'améliorer la performance (temps de réponse, données transférées, etc.)**



PROBLÈME D'ALLOCATION

- Pour assurer l'optimalité, nous devons respecter les deux conditions:
 - 1. Le coût minimal
 - 2. La performance



PROBLÈME D'ALLOCATION

- 1. Le coût minimal

Les coûts traités dans cette condition sont:

- Le coût de stockage de chaque fragment F_i sur le site S_j ;
- Le coût de consultation de F_i localisé sur S_j ;
- Le coût de mise à jour de F_i sur tous les sites où il est stocké;
- Le coût de communication.



PROBLÈME D'ALLOCATION

○ 2. La performance

Dans cette condition nous essayons de:

- Minimiser le temps de réponse;
- Maximiser le débit de chaque site.

○ Remarque:

- Suite à la complexité du problème, on ne peut pas satisfaire toutes ces conditions.
- Les modèles existants essayent de satisfaire les conditions qui les intéressent.



PROBLÈME D'ALLOCATION

- **Autres contraintes**

- o Stockage, équilibrage de la charge entre les sites

- **Donc on a un Problème NP-complet**



ALGORITHMES D'ALLOCATION

- Il existe plusieurs algorithmes, nous présentons ici l'algorithme « **BEST FIT** »
- Principe
 - Permet de déterminer le site sur lequel stocker le fragment, là où l'on maximise les traitements locaux
 - n'intègre pas le problème de la réplication



ALGORITHMES D'ALLOCATION

- Exemple
- Soient les informations suivantes relatives aux différents fragments :

Fragments	temps d'accès local (ms)	temps mise à jour local	temps accès distant	temps m.a.j. distant
R1	100	150	500	600
R2	150	200	650	700
R3	200	250	1000	1100

Transactions	sites	fréquence	accès tables (L= lecture, E = écriture)
T1	S1,S4,S5	1	4 accès R1 (3L, 1E) 2 accès R2 (2L)
T2	S2,S4	2	2 accès R1 (2L) 4 accès R3 (3L, 1E)
T3	S3,S5	3	4 accès R2 (3L, 1E) 2 accès R3 (2L)



ALGORITHMES D'ALLOCATION

- Si on récapitule les deux tableaux dans un seul tableau de synthèse on aura :

Fragment	Site	Accès T1 * Fréquence	Accès T2 * Fréquence	Accès T3 * Fréquence	Total pondéré
R1	S1	4*1	0	0	4
	S2	0	2*2	0	4
	S3	0	0	0	0
	S4	4*1	2*2	0	8
	S5	4*1	0	0	4
R2	S1	2*1	0	0	2
	S2	0	0	0	0
	S3	0	0	4*3	12
	S4	2*1	0	0	2
	S5	2*1	0	4*3	14
R3	S1	0	0	0	0
	S2	0	4*2	0	8
	S3	0	0	2*3	6
	S4	0	4*2	0	8
	S5	0	0	2*3	6



ALGORITHMES D'ALLOCATION

- Selon les données de la colonne « Total pondéré » on a :
- ☐ **On stocke R1 sur S4**
- ☐ **On stocke R2 sur S5**
- ☐ **On stocke R3 sur S2 ou S4 (S2 pour mieux répartir)**

